SIH 2026 · PROBLEM STATEMENT 168 · MODEL REPORT

Smartphone dead reckoning through GNSS outages

A residual Temporal Fusion Transformer cleans the phone's IMU. A 9-state EKF integrates it, and an OSM particle filter keeps it on real roads. This report covers the architecture, the method and where the remaining drift comes from.

Dataset IO-VNBD · 10 Hz · 60 s outages · median drift = end-point error ÷ distance travelled · results as of 30 Sep 2026

9.6%

Median 60 s drift, held-out test drives Y1 + M

3 seeds, range 9.5–9.9 · target < 10%

9.1%

Median 60 s drift, held-out drive Vw4

3 seeds · never used for tuning

28.5% → 18.1%

Open-loop drift, raw IMU vs TFT-corrected IMU

no map, no filter · 888 test windows

492k

Model parameters

3.8 MB ONNX · CPU · edge-deployable

1. The problem

When a phone loses GNSS (tunnels, urban canyons, multi-storey car parks), the app has to keep estimating position from the phone's own accelerometer and gyroscope. Integrating raw phone IMU twice drifts fast. On our held-out test windows it reaches a median of **28.5%** of the distance travelled after 60 s. The problem statement asks for drift below 10%, map matching to real roads, and a model small enough to run on the phone.

We use the IO-VNBD dataset: phone (S) and vehicle (V) CSV logs at 10 Hz with GNSS ground truth. Two data problems had to be fixed before any learning:

- **Clock offsets.** Phone and vehicle clocks disagreed by 0.3–116 s. We re-synchronised each phone log by cross-correlating its gyro yaw rate against GNSS-derived yaw rate.
- **Unknown phone mounting.** The phone's axes aren't aligned with the car. We estimate the rotation per drive by regressing the phone IMU against GNSS-derived vehicle motion.

Drives are split three ways. Training uses most drives. Validation uses **S2 + Vtb5** only: two other validation drives were in the training set of the current checkpoint, so they're excluded from model selection. The

test drives **Y1, M and Vw4** are touched only for final reporting.

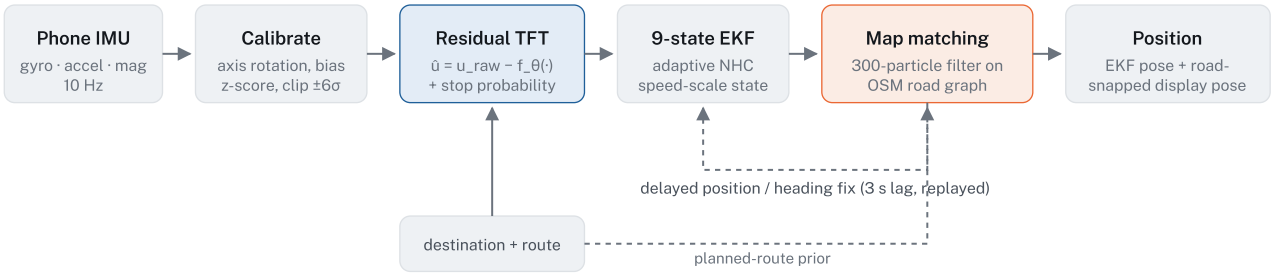
---

2. End-to-end pipeline

Each stage has one job. The network never outputs a position; it outputs corrected sensor readings. Physics and the filters turn those into a trajectory, which keeps the output physically plausible and lets the map correct it.

Data flow during a GNSS outage

Solid arrows run at 10 Hz. Dashed arrows are 1 Hz map feedback applied with a 3 s delay and replayed forward.



The TFT, EKF and map matcher each see the planned route: the network as route geometry features, the particle filter as a prior bonus on route edges. All three are optional, so the pipeline degrades gracefully when no destination is set.

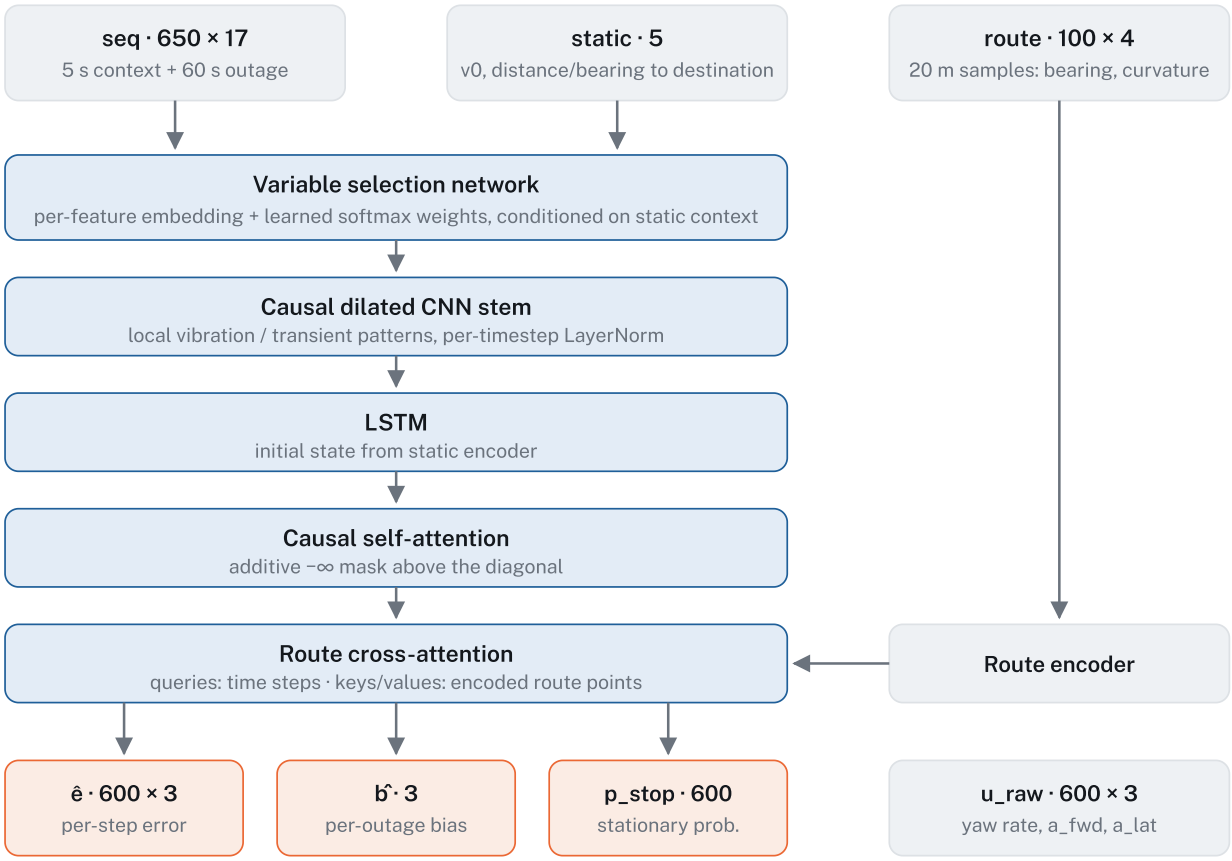
---

3. Model architecture

The network is a **residual** model. It predicts the phone's sensor error, which is then subtracted from the raw reading. It doesn't predict velocity or position directly. A model that only has to learn corrections starts from a sensible answer (the raw IMU) and generalises better than one that has to learn motion from scratch.

ResidualTFT, non-autoregressive, whole 60 s outage at once

Input shapes for one sample. Every temporal layer is causal: step k never sees data after k.



`û = u_raw - (ê + b̂) → physics integration`

Why non-autoregressive: the whole outage is predicted in one forward pass, so errors in early predictions are never fed back as inputs. That removes exposure bias and the need for scheduled sampling. **Why causal:** at run time the phone only has data up to "now", so the model is trained under the same constraint, and future steps are zero-padded.

Physics integration (same equations in training and in the app)

```
Δψ_k = -Σ_{j≤k} ω̂_j · Δt
v_k  = relu( v0 + Σ_{j≤k} â_fwd,j · Δt ) · (1 - p_stop,k)
p_k  = p0 + Σ v_k · [sin ψ_k, cos ψ_k] · Δt          Δt = 0.1 s
```

Because integration is differentiable, the losses are computed on the resulting trajectory as well as on the corrected sensor values.

Training objective

- **Velocity-direction loss:** minimises the angle between predicted and true velocity vectors, penalising heading drift where it originates.
- **Increment losses** on per-step heading change and speed change, weighted by sample (`w_speed`) so braking and turning segments aren't drowned out by cruising.
- **Speed magnitude** and **stop** losses (`p_stop` against zero-speed labels).
- **Residual regularisation:** keeps ê small and smooth, so the model corrects the sensor rather than inventing motion.

Each training window gets a **random destination** taken from later in the same drive, with the route between them, so the model learns to use a destination when one exists and to cope without one. On test windows, route input lowers open-loop drift from 18.9% (destination only) to 18.1%.

---

4. Filtering and map matching

The TFT output is integrated by a 9-state planar EKF. It carries what the network can't know from a single window: the slowly varying biases and the accumulated uncertainty.

STATE	MEANING	ROLE
`E, N`	Position (east, north)	What the app shows
`ψ`	Heading	Integrated from corrected yaw rate
`v_f, v_l`	Forward and lateral speed	v_l ≈ 0 enforced by the non-holonomic constraint (NHC)
`b_g, b_a`	Gyro and accel bias	Made observable by road-heading updates
`ds, b_s`	Speed scale and offset	Absorbs systematic speed error

Adaptive NHC. A car doesn't slide sideways, so lateral speed is pseudo-measured as zero. The noise on this constraint grows with yaw rate, acceleration and vibration, with Student-t reweighting of outliers, so it's trusted on straight roads and relaxed in hard turns.

Map matching. 300 particles live on the OSM road graph extracted per drive. Each particle has its own road, arc position, lateral offset and speed-scale error. Once a second each is weighted by:

- cross-track distance to its road;
- along-track agreement with the EKF;
- heading against the road bearing;
- curvature against the gyro;

- a large penalty for travelling against a road's direction.

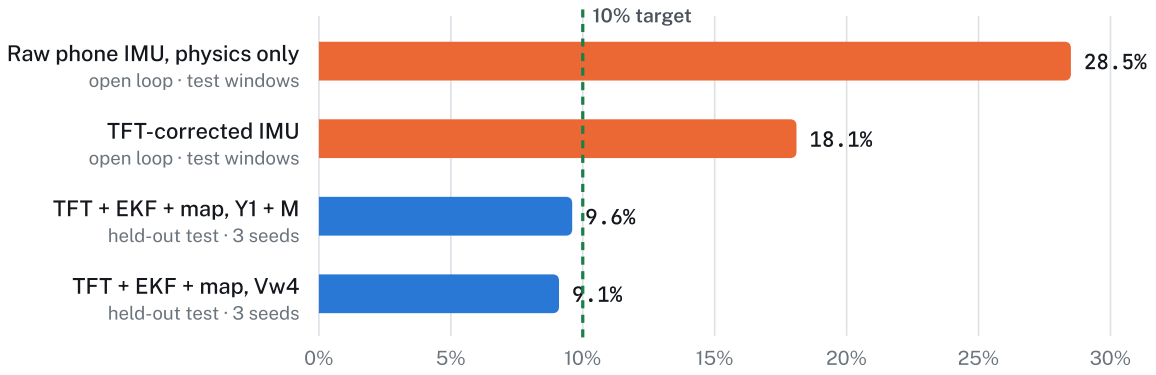
At junctions particles branch in proportion to heading agreement, with a planned-route bonus. The filter decides with a **3 s fixed lag**. When confident, it sends a position (and on straight roads, heading) fix back to the EKF, applied at the lagged time and replayed to the present.

---

5. Drift results

Median 60 s drift at each stage

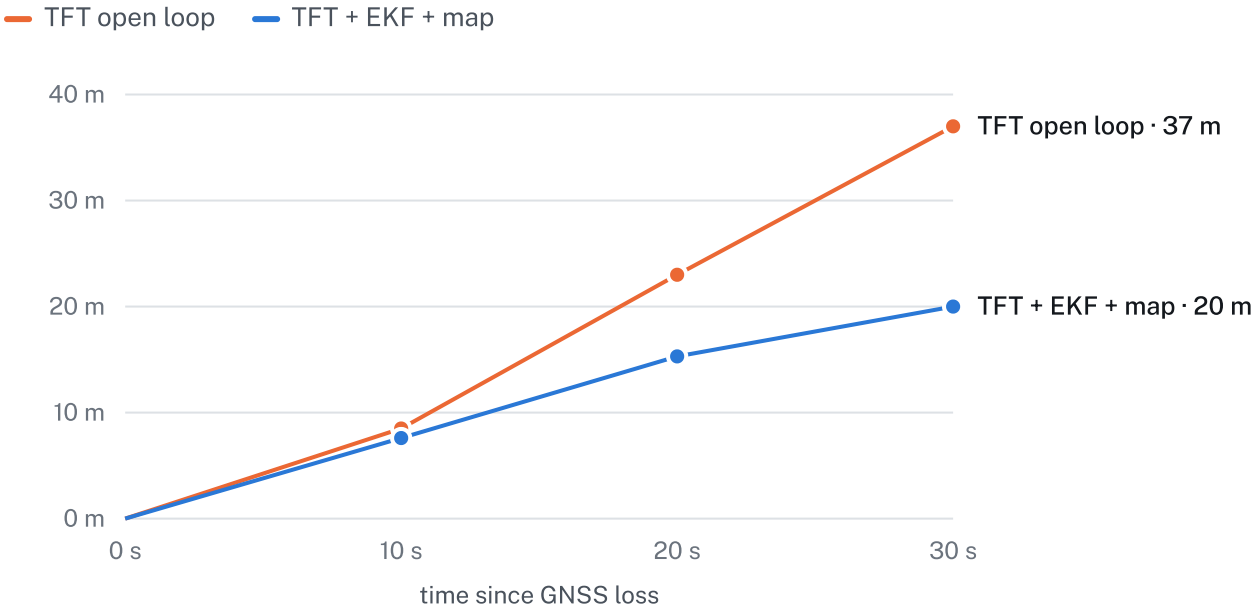
Lower is better. Target: below 10% of distance travelled.



Open-loop rows: 888 held-out test windows. Map-aided rows: outages whose ground truth stays within 15 m of a mapped road (268 for Y1 + M, 175 for Vw4), mean of 3 particle-filter seeds. Map-aided accuracy depends on road coverage, and off-map outages are reported separately.

How error grows during an outage, Y1 + M

Median position error in metres after 10, 20 and 30 s.



Without the map, error grows almost linearly with time: a speed error integrates into a distance error. Map matching removes most cross-track error and bends the curve after the first junction.

SPLIT (60 S OUTAGES)	MEDIAN DRIFT	SEED RANGE	P90	< 10% DRIFT	WRONG ROAD	ALONG / CROSS
Test Y1 + M	9.6%	9.5–9.9	50%	51%	14.9%	44 / 7 m
Test Vw4 (held out)	9.1%	8.6–9.7	40%	55%	14.7%	79 / 6 m
Validation S2 + Vtb5	9.4%	8.6–9.9	45%	52%	15.6%	47 / 5 m

> The median meets the target, but only about half of all outages are individually under 10%, and the worst tenth drift 40–50%. The rest of this report is about that tail.

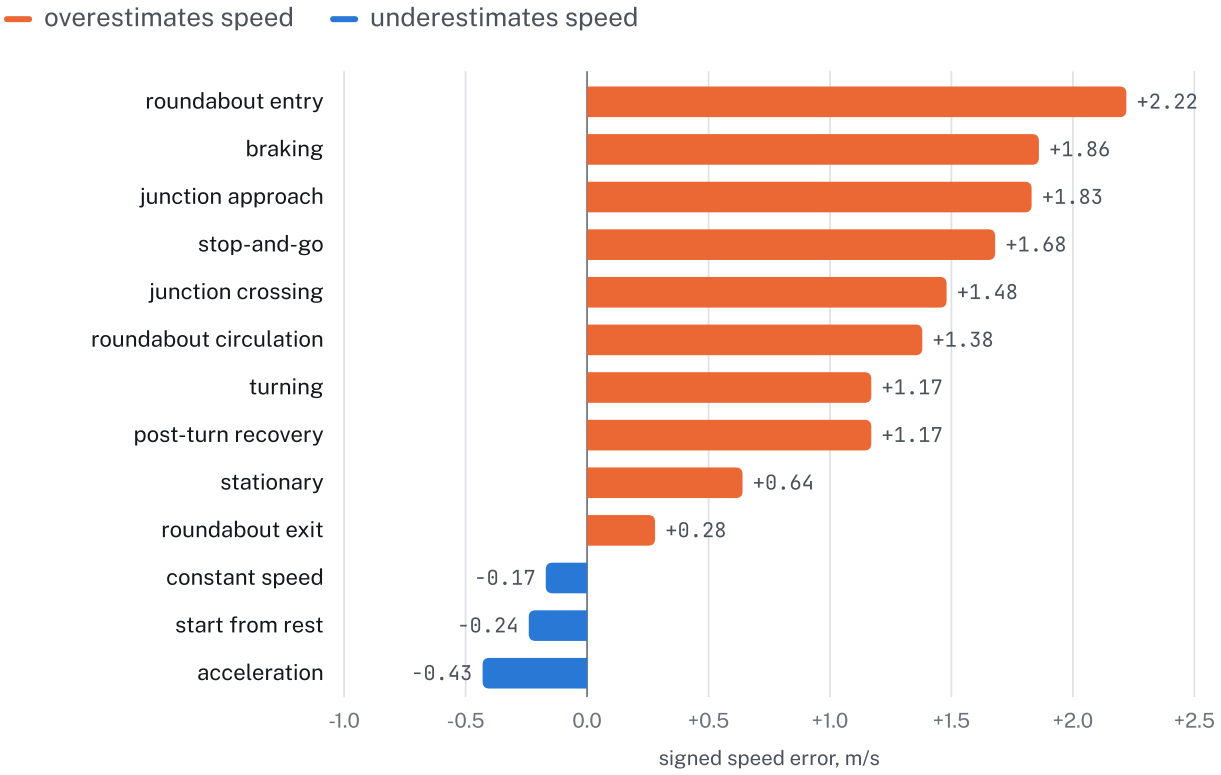
6. Where the remaining drift comes from

Once the map fixes cross-track error, almost all remaining error is **along-track**: the car is on the right road but the estimate is ahead of or behind it. The median along-track error is about 45 m, against 5–7 m cross-

track. Along-track error is speed error integrated over time, so we sliced the TFT's speed error by driving condition on the validation drives.

TFT speed bias by motion phase

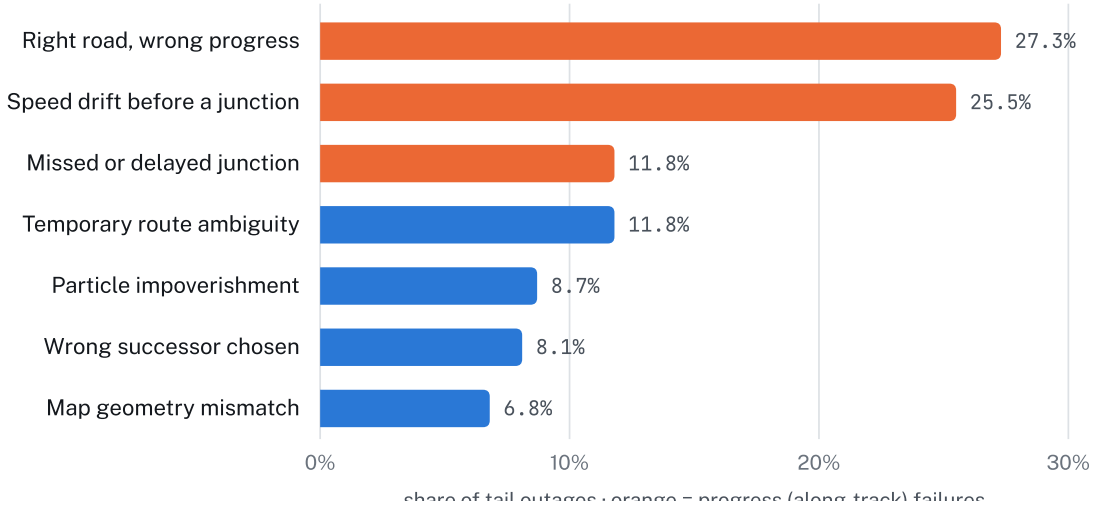
Mean signed speed error (m/s), validation drives. Positive means the model thinks the car is faster than it is.



Diagnosis: the model is accurate at constant speed (−0.17 m/s) but responds too slowly to deceleration. It keeps too much speed when braking, approaching junctions and entering roundabouts, lagging the truth by 0.2–1.5 s. Across speeds it regresses to a typical value: +2.0 m/s at 3–8 m/s, −0.5 m/s above 15 m/s. Vibration, jerk and timestamp quality made little difference.

What goes wrong in the worst quarter of outages

Validation S2 + Vtb5: 161 tail outages, each assigned one failure class.



About two thirds of the tail is a progress error: the right road but the wrong place along it, often compounded at the next junction. In 74% of tail outages along-track error appears first, a median 15 s before any wrong turn. Wrong turns are mostly a *consequence* of speed drift, not an independent failure.

7. What we tried, and what we kept

Every change is an ablation against the same control on the validation drives, with 3 particle-filter seeds. It's kept only if it improves drift without worsening the tail or wrong-road rate. Seed-to-seed noise is about ±0.3–1 point, so small gains are treated as no change.

CHANGE	EFFECT	DECISION
Heading-only map updates (config B)	8.6% on Y1 + M, but 10.9% on the unseen Vw4	rejected, overfit
Speed calibration (affine / isotonic)	small median gain, worse tail	flag, off
Deceleration-weighted retrain (D1)	braking bias +1.9 → +0.5 m/s, but cruising speed now too low; map drift 11.1%	rejected
Multitask roundabout model	worse map-aided drift	rejected
Turn landmark v1: particle turn records	no change; 81% of turns skipped	flag, off
Turn landmark v2: turn located on the map	wrong-road 15.6 → 14.7%, wrong successor 22 → 19, but worst-quarter along-track 141 → 155 m	needs gating
Confidence-gated map policy (A3)	implemented and unit-tested; full validation run pending	in progress

Next: confidence-aware map corrections

The v2 turn landmark shows the map holds the information needed to fix progress errors, but applied blindly it also makes some outages worse. The confidence layer computes three separate scores:

- **dead reckoning** (EKF covariance, speed-scale uncertainty, NHC innovation, TFT residual);
- **map match** (cross-track, along-track and heading residuals kept separate, particle effective sample size, cloud spread);
- **decision** (best vs second-best road probability).

These map to four actions. High confidence allows the along-track junction fix. Medium confidence softens and clips the fix. Low confidence lets the map only rule out impossible roads, with no snapping. Very low confidence widens the particle search and waits for the next turn.

In unit tests, the policy keeps both hypotheses alive on genuinely ambiguous parallel roads 82% of the time, against 55% for the current filter. It changes nothing when switched off: tracks are identical to the validated pipeline on real outages.

8. Deployment

- Exported to ONNX (opset 17, 3.8 MB), with a manifest listing input layout, feature order, normalisation statistics and filter configuration.
- **ONNX Runtime must use `ORT_ENABLE_BASIC`.** The higher optimisation levels contain an operator-fusion bug (onnxruntime 1.29) that corrupts this model's outputs by up to 0.6. With BASIC, outputs match PyTorch within 2.3×10^{-6}.
- The app asserts the optimisation level at startup and runs four stored real windows as a self-check. A CI test compares PyTorch and ONNX on random and real inputs.
- The EKF and particle filter are plain NumPy-level arithmetic (300 particles, 1 Hz map step) and port directly to the app.

9. Limitations

- Map-aided figures cover outages on mapped roads. Off-map driving (car parks, private roads) falls back to TFT + EKF, which is about 18% open loop.
- The test set is three drives, so seed-to-seed and drive-to-drive variation (±0.3–1 point) is similar in size to many candidate improvements.
- Normalisation statistics differ slightly from those the checkpoint was trained with (two validation drives were included then). The effect is small and documented.
- The median target is met; the tail (p90 of 40–50%) is not yet good enough for turn-by-turn guidance through long outages.